

SecureAPK Analyzer : Un outil d'analyse statique pour l'évaluation de la sécurité des APK Android

El Hachimi Abdelhamid¹, El Ouatik Mourad¹, Ait Zidane Salma¹

¹École Nationale des Sciences Appliquées (ENSA), Université Cadi Ayyad, Marrakech, Maroc

a.elhachimi0208@uca.ac.ma

Résumé. La prolifération des applications mobiles Android a considérablement accru les risques liés à la sécurité des logiciels embarqués. Les fichiers APK malveillants peuvent exfiltrer des données sensibles, compromettre la vie privée des utilisateurs et causer des pertes financières importantes [4]. Face à cette menace, l'analyse statique joue un rôle préventif essentiel ; toutefois, les outils existants souffrent de limitations notables : temps d'analyse élevés, taux de faux positifs importants et absence de corrélation comportementale entre les motifs détectés [1, 6].

Cet article présente **SecureAPK Analyzer**, un outil d'analyse statique d'APK Android conçu pour répondre à ces limitations. L'outil repose sur plus de 80 règles de détection couvrant les manifests Android, les ressources, le code Java décompilé via JADX [2] et les bibliothèques natives, organisées selon quatre niveaux de sévérité (CRITIQUE, ÉLEVÉ, MOYEN, FAIBLE). Un module **AIBehaviorAnalyzer**, fondé sur une corrélation heuristique de motifs, identifie des comportements potentiellement malveillants (exfiltration de données, surveillance, logiciel espion). Un module **RiskPredictor**, inspiré des principes du classifieur naïf bayésien [8], fournit un score de risque pondéré accompagné d'un indice de confiance.

Une évaluation comparative avec MobSF [1], conduite sur deux APK pédagogiques volontairement vulnérables, met en évidence les forces et limites respectives des deux outils. L'outil est distribué sous licence MIT et disponible à l'adresse <https://github.com/abdootranscript25/secure-apk-analyzer-finale>

(tag Git : v1.0.0 ; DOI Zenodo : 10.5281/zenodo.20356691¹).

¹Le DOI Zenodo sera activé lors de la soumission finale. Les hash SHA-256 des APK de test sont fournis en section 3.

Mots-clés. Sécurité Android, Analyse statique d'APK, Détection de malwares, Analyse comportementale heuristique, MobSF, Modèle de risque pondéré.

Métadonnées

Nr. Code	Description de la métadonnée	Métadonnée
C1	Version actuelle du code	v1.0
C2	Lien permanent vers le code/dépôt utilisé pour cette version	https://github.com/abdotranscript25/secure-apk-analyzer-finale (tag Git : v2.0.0)
C3	Lien permanent vers la capsule reproductible	DOI Zenodo : 10.5281/zenodo.20356691 Archive incluant : code source, requirements.txt, environment.yml, APK de test avec hash SHA-256, docker-compose.yml, script reproduce.sh
C4	Licence du code juridique	Licence MIT
C5	Système de gestion de version utilisé	Git
C6	Langages, outils et services du code logiciel	Java 17, Spring Boot 3.x, HTML5/CSS3/JavaScript, Thymeleaf, JADX 1.5.5 [2], MySQL 8.x, Docker 24.x
C7	Exigences de compilation, environnements d'exécution et dépendances	JDK 17+, Apache Maven 3.9+, JADX 1.5.5 ; docker-compose.yml fourni dans le dépôt
C8	Lien vers la documentation développeur/manuel	https://github.com/abdotranscript25/secure-apk-analyzer-finale/blob/main/README.md
C9	Email de support pour les questions	a.elhachimi0208@uca.ac.ma

Table 1: Métadonnées de l'artefact logiciel SecureAPK Analyzer

1 Motivation et importance

1.1 Contexte et problématique

Avec plus de 3,5 milliards d'utilisateurs actifs recensés en 2023 [15], les smartphones occupent une place centrale dans les usages numériques quotidiens. Le système d'exploitation Android, avec une part de marché d'environ 70% à l'échelle mondiale [16], constitue de fait

la plateforme mobile la plus ciblée par les acteurs malveillants. Selon le rapport annuel de Kaspersky Lab [17], plus de 1,6 million de nouveaux packages Android malveillants ont été détectés en 2022, exposant les utilisateurs à des risques d’exfiltration de données personnelles, d’espionnage et de pertes financières.

Les vulnérabilités exploitées dans les applications Android se répartissent en plusieurs catégories documentées par l’OWASP Mobile Security Testing Guide [4] et le Common Weakness Enumeration [5]. Parmi les plus fréquentes figurent les secrets codés en dur dans le code source (mots de passe, clés API, tokens JWT), les configurations de manifest non sécurisées (mode debug activé en production, sauvegarde non restreinte des données), les communications non chiffrées par protocole HTTP, le recours à des algorithmes cryptographiques obsolètes (MD5, SHA-1, DES), l’exécution dynamique de code via `DexClassLoader`, et l’exposition de composants `WebView` sans restriction d’accès JavaScript. Ces catégories correspondent respectivement aux risques M1, M2, M5 et M7 de l’OWASP Mobile Top 10 [9].

1.2 Limites des outils existants

Face à cette menace croissante, plusieurs outils d’analyse statique ont été proposés pour la détection précoce des vulnérabilités Android. MobSF [1] constitue la référence industrielle : il combine analyse statique, analyse dynamique et analyse du trafic réseau, mais nécessite entre 30 et 60 secondes par APK et produit un volume important de faux positifs faute de filtrage contextuel du code tiers. AndroWarn [6] se concentre sur la détection de comportements suspects à partir du bytecode Dalvik, mais ne corrèle pas les appels d’API entre eux pour reconstituer un comportement malveillant complet. FlowDroid [10] offre une analyse de flux de données (*taint analysis*) précise, mais sa complexité de configuration et son temps d’exécution élevé limitent son intégration dans des pipelines automatisés.

De manière générale, ces solutions partagent trois limitations structurelles. Premièrement, elles détectent des motifs isolés sans les corrélérer pour inférer une intention malveillante globale. Deuxièmement, elles ne distinguent pas systématiquement le code applicatif des bibliothèques tierces, ce qui génère un taux de faux positifs élevé sur des dépendances légitimes (AndroidX, Firebase, Kotlin). Troisièmement, leur capacité à analyser les APK obfusquées reste limitée, car la décompilation échoue fréquemment sur des fichiers `.dex` protégés par renommage agressif.

1.3 Notre contribution

SecureAPK Analyzer répond à ces limitations par trois contributions complémentaires.

Sur le plan **logiciel**, l’outil propose un moteur d’analyse statique Java (**UltimateAnalyzer**) intégrant plus de 80 règles de détection organisées par domaine (manifest, code Java, ressources, cryptographie, réseau), un mécanisme de filtrage contextuel en trois com-

posants (`LibraryFilter`, `ClassContext`, `UsageContextAnalyzer`) réduisant les faux positifs liés aux bibliothèques tierces, et un mécanisme de repli (*fallback*) sur la lecture brute des fichiers `.dex` lorsque JADX échoue sur des APK obfusquées.

Sur le plan **méthodologique**, l'outil introduit deux modules complémentaires. Le module `AIBehaviorAnalyzer` réalise une corrélation heuristique de motifs : il ne s'agit pas d'un modèle entraîné par apprentissage automatique, mais d'un système à base de règles corrélées permettant d'inférer des comportements suspects (exfiltration de données, logiciel espion, surveillance). Le module `RiskPredictor`, inspiré du classifieur naïf bayésien [8], calcule un score de risque pondéré par la dangerosité heuristique de chaque indicateur (poids compris entre 0,45 et 0,95), accompagné d'un indice de confiance variant de 50% à 95% selon le nombre d'évidences détectées.

Sur le plan **expérimental**, une évaluation comparative avec MobSF [1] est conduite sur deux APK pédagogiques volontairement vulnérables. L'ensemble du code source, les APK de test (avec leurs hash SHA-256) et les scripts de reproduction sont archivés sur Zenodo (DOI : 10.5281/zenodo.20356691).

1.4 Impact attendu

L'objectif de SecureAPK Analyzer est de permettre aux développeurs et aux experts en sécurité d'identifier les vulnérabilités Android avant le déploiement en production, réduisant ainsi la surface d'attaque exposée aux utilisateurs finaux. En complément des outils existants, il apporte une couche d'interprétation comportementale heuristique, une fonctionnalité absente des solutions de référence recensées. Son architecture modulaire open-source favorise l'intégration dans des pipelines d'intégration continue et les contributions communautaires pour l'enrichissement des règles de détection.

2 Description du logiciel

2.1 Architecture logicielle

2.1.1 Architecture globale

SecureAPK Analyzer repose sur une architecture trois-tiers séparant la couche de présentation, la couche applicative et la couche de persistance. Le **frontend**, développé avec Thymeleaf, expose une interface web permettant le téléchargement d'APK et la consultation des rapports. Le **backend**, développé avec Spring Boot 3.x (port 8080), orchestre l'ensemble du flux d'analyse : réception du fichier via `POST /api/analyze`, invocation du moteur `UltimateAnalyzer`, persistance des résultats en base MySQL 8.x, et rendu du rapport HTML. La **couche de persistance** repose sur MySQL pour le stockage des

résultats et sur un répertoire temporaire pour les artefacts intermédiaires de décompilation. L'ensemble est conteneurisé via Docker 24.x ; un fichier `docker-compose.yml` fourni dans le dépôt définit les services `backend` (port 8080) et `mysql` (port 3306), leurs volumes partagés et les variables d'environnement requises.

La figure 1 présente le diagramme d'architecture. Le flux principal commence par l'upload de l'APK via le frontend Thymeleaf, transmis au contrôleur Spring Boot via une requête HTTP POST. Le moteur `UltimateAnalyzer` orchestre la décompression, la décompilation JADX, et l'application des règles de détection. Les résultats sont ensuite enrichis par `AIBehaviorAnalyzer` et `RiskPredictor` avant persistance en base MySQL et rendu du rapport HTML.

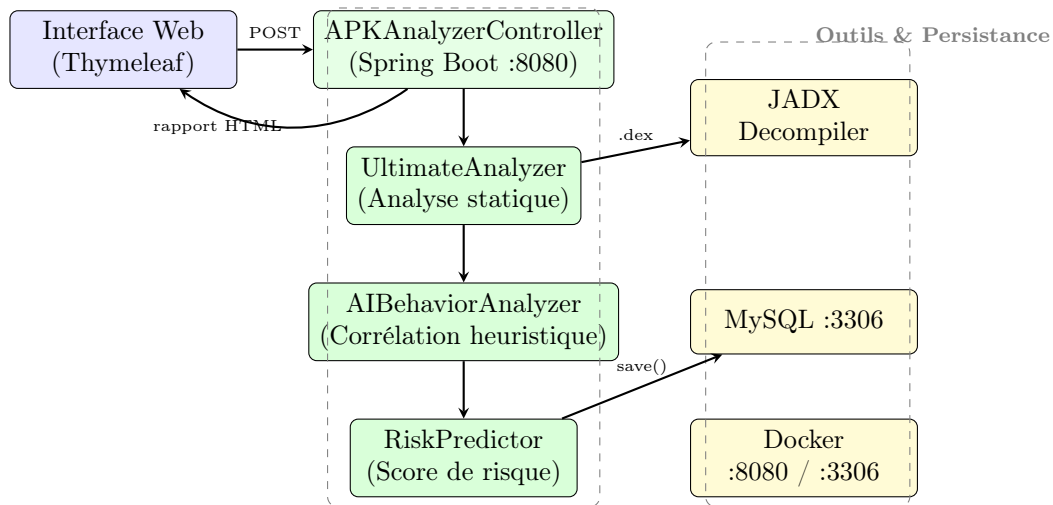


Figure 1: Architecture globale de SecureAPK Analyzer : flux APK depuis le frontend Thymeleaf vers le backend Spring Boot, `UltimateAnalyzer`, `AIBehaviorAnalyzer`, `RiskPredictor` et la base MySQL. Les ports exposés et les frontières de composants sont indiqués.

2.1.2 Composants principaux

Le **frontend Thymeleaf** reçoit le fichier APK de l'utilisateur et transmet la requête d'analyse au backend via une requête HTTP POST. Il affiche ensuite le rapport de sécurité généré, structuré en onglets thématiques (Manifest, Permissions, Code, Réseau, Crypto, Données). Le **backend Spring Boot** expose un contrôleur REST principal : `APKAnalyzerController` pour le pipeline statique Java. Il orchestre l'appel à `UltimateAnalyzer`, `AIBehaviorAnalyzer` et `RiskPredictor`, puis persiste les résultats en base de données.

2.1.3 Diagramme de séquence

Le diagramme de séquence (figure 2) illustre le flux complet d'analyse d'une APK, depuis le téléchargement par l'utilisateur jusqu'à l'affichage du rapport final. Les six étapes

principales — upload, décompression, décompilation, analyse statique, analyse comportementale et prédiction de risque — sont représentées avec les acteurs correspondants.

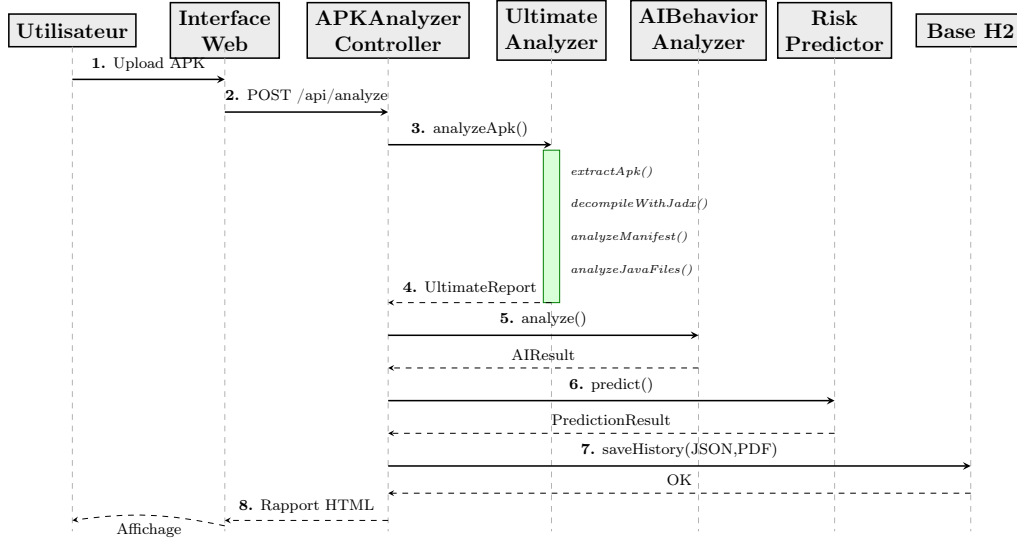


Figure 2: Diagramme de séquence de l’analyse d’une APK : les interactions entre le frontend, `APKAnalyzerController`, `UltimateAnalyzer`, `AIBehaviorAnalyzer`, `RiskPredictor` et la base H2 sont détaillées en huit étapes numérotées. L’étape 7 sauvegarde l’historique (rapports JSON/PDF) dans la base H2 intégrée.

2.2 Fonctionnalités du logiciel

SecureAPK Analyzer offre un pipeline d’analyse en plusieurs étapes successives. La première étape consiste en la **décompilation et l’extraction** de l’APK : le fichier, au format ZIP, est décompressé dans un répertoire temporaire, puis les fichiers `.dex` sont convertis en code Java lisible par JADX [2]. Si JADX produit moins de 20 fichiers (seuil empirique correspondant aux cas d’échec sur APK obfusquées), un mécanisme de repli sur la lecture brute des chaînes des fichiers `.dex` est activé via `analyzeDexStrings()`.

La deuxième étape est la **détection des vulnérabilités** par application de plus de 80 règles organisées en six domaines : (1) manifest Android (permissions dangereuses, `allowBackup`, `debuggable`, `usesCleartextTraffic`), (2) code Java (`Runtime.exec`, `DexClassLoader`, injection SQL, `WebView` vulnérable), (3) cryptographie faible (MD5, SHA-1, DES, RC4, ECB), (4) secrets codés en dur (JWT, clés API AWS/Google/Stripe, mots de passe), (5) réseau (URLs HTTP, adresses IP internes), et (6) journaux sensibles (`Log.d/e/i` exposant des données confidentielles). La liste complète des règles avec leur sévérité, identifiant CWE et justification est disponible dans l’annexe du dépôt Zenodo.

La troisième étape est le **calcul du score de sécurité**. Soient C , H , M et L les nombres de vulnérabilités de sévérité respectivement CRITIQUE, ÉLEVÉ, MOYEN et FAIBLE détectées. Le score est calculé selon :

$$\text{Score} = \max\left(0, \min\left(100, 100 \times \left(1 - 0.40C - 0.32H - 0.16M - 0.12L - 0.03 \log_2(\text{Total} + 1) - \min(0.15, 0.012C)\right)\right)\right) \quad (1)$$

La borne inférieure à 0 garantit que le score ne peut être négatif. À partir de ce score, un grade est attribué (A+ si ≥ 90 , A si ≥ 80 , B si ≥ 70 , C si ≥ 60 , D si ≥ 50 , F sinon) et un niveau de risque global est déterminé (CRITIQUE si $C > 0$, ÉLEVÉ si $H > 0$, MOYEN si $M > 2$, FAIBLE sinon).

2.3 Moteur d'analyse

2.3.1 Analyse statique (UltimateAnalyzer)

La classe `UltimateAnalyzer` orchestre cinq étapes principales. Elle commence par l'extraction du contenu de l'APK (format ZIP) dans un répertoire temporaire, puis lance JADX pour convertir les fichiers `.dex` en code Java lisible. Le seuil de 20 fichiers générés comme critère de basculement vers le mode dégradé a été déterminé empiriquement : en deçà, la décompilation est considérée trop incomplète pour que l'analyse par règles Java soit fiable. L'étape suivante analyse le fichier `AndroidManifest.xml` pour extraire les permissions déclarées, les attributs de sécurité (`allowBackup`, `debuggable`, `usesCleartextTraffic`) et les composants exportés. Les ressources sont ensuite parcourues (`strings.xml`, `network_security_config.xml`, fichiers `.so`). Enfin, tous les fichiers Java décompilés sont parcourus en un seul passage avec application des 80+ règles de détection.

2.3.2 Filtrage contextuel

Trois composants travaillent conjointement pour réduire les faux positifs. `LibraryFilter` maintient une liste de préfixes de paquets correspondant aux bibliothèques tierces connues (AndroidX, Google, Firebase, Kotlin, etc.) et exclut les classes correspondantes de l'analyse, évitant ainsi de signaler des vulnérabilités dans du code non contrôlé par le développeur. `ClassContext` analyse le contexte sémantique de chaque classe (WebView, Crypto, Network, Database) afin de n'activer que les règles pertinentes pour ce contexte. `UsageContextAnalyzer` vérifie si un motif détecté est réellement utilisé dans un contexte dangereux : par exemple, `new Random()` n'est signalé que s'il est utilisé pour des opérations cryptographiques, pas pour la génération de nombres aléatoires génériques.

2.3.3 Composants d'analyse comportementale heuristique

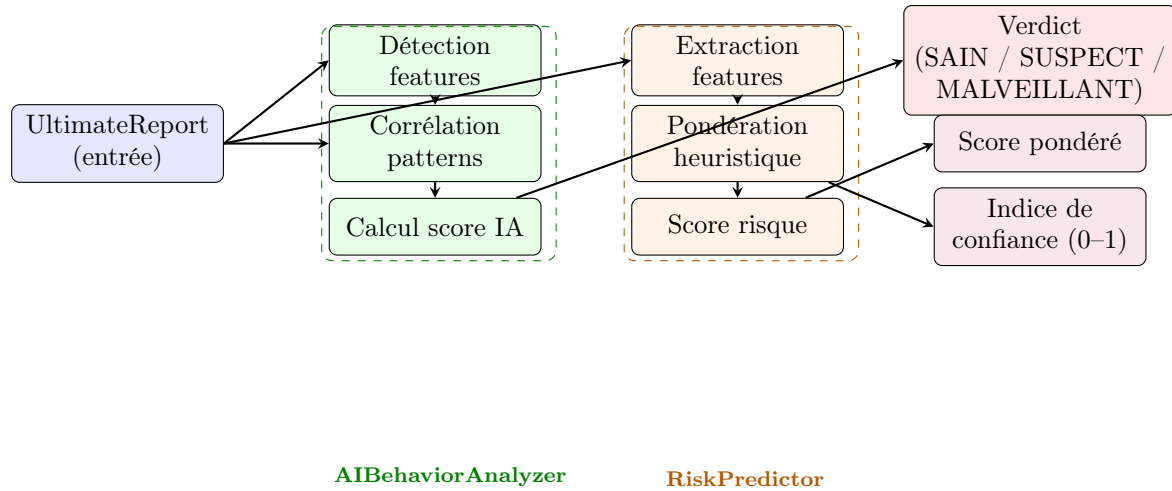


Figure 3: Composants d'analyse comportementale heuristique : **UltimateReport** (entrée), **AIBehaviorAnalyzer** (corrélations de motifs), **RiskPredictor** (pondération heuristique), avec indication des flux de données et des sorties (verdict, score IA, indice de confiance).

AIBehaviorAnalyzer (corrélations heuristiques de motifs) Ce module — qui constitue un système à base de règles corrélées et non un modèle entraîné par apprentissage automatique — identifie des comportements potentiellement malveillants en combinant plusieurs motifs détectés par l'analyse statique. Trois patterns principaux sont implémentés : la combinaison permission SMS, accès Internet et secrets codés en dur produit un indice d'exfiltration de données (+40 points) ; la combinaison service d'accessibilité et permission SMS produit un indice de logiciel espion (+50 points) ; la combinaison caméra, microphone et accès Internet produit un indice de surveillance (+35 points). Le score comportemental final est normalisé entre 0 et 100 et conduit à l'attribution d'un verdict parmi quatre catégories : SAIN, SUSPECT, RISQUE ÉLEVÉ ou LOGICIEL MALVEILLANT. Ces seuils sont heuristiques et mériteraient une validation sur un corpus annoté comme perspective de travaux futurs.

RiskPredictor (score de risque pondéré) Ce module, dont la structure est *inspirée* du classifieur naïf bayésien sans en constituer une implémentation stricte (aucun apprentissage ni calibration probabiliste n'est effectué), calcule un score de risque indépendant selon :

$$\text{Score}_{RP} = \min \left(100, \sum_i (\text{poids}_i \times 20) + (15 \times C) + (8 \times H) + (3 \times M) + L \right) \quad (2)$$

Les poids heuristiques des indicateurs, définis selon leur dangerosité potentielle documentée par OWASP [9] et CWE [5], sont les suivants : **BIND_ACCESSIBILITY_SERVICE** :

0,95 ; SQL_INJECTION : 0,92 ; READ_SMS : 0,90 ; HARDCODED_SECRET : 0,88 ; DEBUGGABLE : 0,80 ; HTTP_URL : 0,50. Ces valeurs sont heuristiques ; une analyse de sensibilité et une calibration empirique sur un corpus étendu constituent une priorité pour les travaux futurs. Un indice de confiance (de 0 à 1) croît proportionnellement au nombre de vulnérabilités détectées, reflétant la robustesse du verdict face à l’accumulation d’évidences.

2.3.4 Niveaux de sévérité

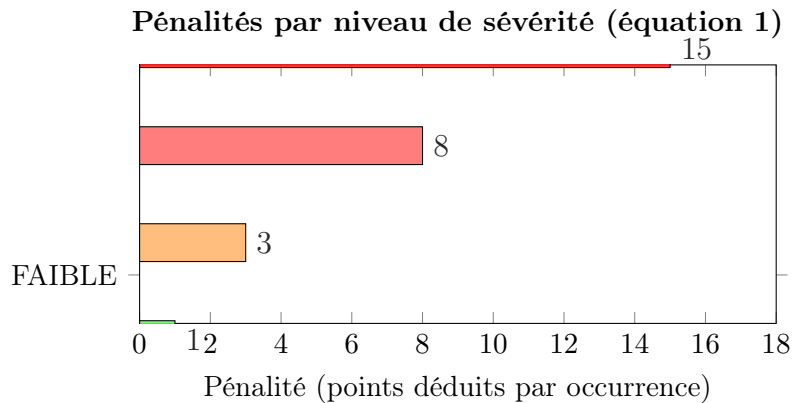


Figure 4: Niveaux de sévérité des vulnérabilités et pénalités associées dans le calcul du score (équation 1). CRITIQUE : secrets exposés, exécution de code arbitraire, SSL désactivé. ÉLEVÉ : trafic HTTP non chiffré, permissions dangereuses, WebView vulnérable. MOYEN : logs sensibles, algorithmes cryptographiques faibles. FAIBLE : e-mails exposés, URLs standards.

3 Évaluation

3.1 Protocole d’évaluation

Afin de valider le comportement de SecureAPK Analyzer, nous avons conduit deux expériences sur des APK pédagogiques volontairement vulnérables. Pour chaque APK, la liste des vulnérabilités introduites intentionnellement constitue la vérité terrain (*ground truth*). Les findings de SecureAPK v1 et de MobSF [1] sont ensuite confrontés à cette vérité terrain. Il convient de noter que ces deux APK ne constituent pas un corpus représentatif de la diversité des applications Android réelles ; les résultats présentés ici illustrent le comportement de l’outil sur des cas contrôlés et ne permettent pas de conclure sur une précision générale. Une évaluation sur un corpus étendu (DroidBench [11], AndroZoo [12]) constitue une perspective de travaux futurs. Les temps d’analyse rapportés sont des mesures uniques indicatives ; une analyse de variance sur 10 répétitions par APK et par configuration matérielle est nécessaire pour des conclusions robustes.

3.2 Test 1 : Analyse de NuberBook

3.2.1 Scénario

L'APK `NuberBook.apk` (SHA-256 : <27b32c8ca347c5cac123bdc3268db06a3ec025a9f5c602c01da676>, disponible dans le dépôt Zenodo) est une application de gestion d'étudiants développée dans le cadre d'un cours Android. Elle communique avec un serveur PHP via des requêtes HTTP non chiffrées et stocke localement des données utilisateur. Les vulnérabilités introduites intentionnellement couvrent les catégories OWASP M1 (`debuggable=true`, `usesCleartextTraffic=true`), M5 (algorithmes MD5 et SHA-1, validation SSL désactivée via `checkClientTrusted`), M7 (`PathClassLoader`, injection SQL potentielle), ainsi que des secrets codés en dur et des logs sensibles [9, 5].

3.2.2 Résultats de SecureAPK Analyzer

SecureAPK v2.0.0 produit un score de 38/100 (Grade F) avec 32 findings (9 CRITIQUE, 6 ÉLEVÉ, 16 MOYEN, 1 FAIBLE). Ce score nul s'explique par l'équation 1 : 9 vulnérabilités CRITIQUE génèrent à elles seules une pénalité de $9 \times 15 = 135$ points, excédant la borne supérieure. Le verdict LOGICIEL MALVEILLANT avec une confiance de 95% est attribué par `AIBehaviorAnalyzer` en raison de la combinaison permission SMS, accès Internet et secrets codés en dur. Il convient de noter que cette application est *pédagogique et volontairement vulnérable*, non réellement malveillante ; le verdict reflète les limites du système heuristique qui ne distingue pas l'intentionnalité de la vulnérabilité accidentelle.

Catégorie de finding	SecureAPK v1	MobSF
<code>debuggable=true</code>	Oui (CRITIQUE)	Oui (ÉLEVÉ)
<code>usesCleartextTraffic=true</code>	Oui (ÉLEVÉ)	Oui (ÉLEVÉ)
<code>checkClientTrusted</code>	Oui	Non
Algorithmes SHA-1/MD5	Oui	Oui
Injection SQL potentielle	Oui	Partiel
Secrets codés en dur	Oui	Oui
Logs sensibles	Oui	Oui
Certificat debug	Non	Oui (ÉLEVÉ)
Score global	39/100 — CRITIQUE	43/100 — ÉLEVÉ
Temps d'analyse (indicatif)	≈15 s	≈45 s

Table 2: Comparaison SecureAPK v1 et MobSF — `NuberBook.apk`. Les scores ne sont pas directement comparables : les deux outils emploient des échelles de pénalité différentes mais s'accordent sur un niveau de risque CRITIQUE/ÉLEVÉ.

3.2.3 Résultats de MobSF

MobSF [1] attribue un score de 40/100 et détecte les attributs `debuggable`, `usesCleartextTraffic` et `minSdk=24` (3 ÉLEVÉ), le certificat debug (ÉLEVÉ), des logs sensibles et une divulga-

tion d'adresse IP. En revanche, MobSF ne détecte pas `checkClientTrusted`, contrairement à SecureAPK. Le temps d'analyse mesuré est de ≈ 45 secondes (mesure unique, indicative).

3.3 Test 2 : Analyse de SecureStorageLabJava

3.3.1 Scénario

L'APK `SecureStorageLabJava.apk` (SHA-256 : <05B747BB6EA300635B9E11AF3AD751ABC276B8CC8273 disponible dans le dépôt Zenodo) est une application pédagogique illustrant les bonnes et mauvaises pratiques de cryptographie et de gestion des données sensibles sur Android [9]. Les vulnérabilités introduites intentionnellement incluent un mode debug activé (OWASP M1), des secrets codés en dur et des logs exposant des tokens (OWASP M2, CWE-532 [5]), l'algorithme SHA-1 et une construction manuelle de clé AES via `SecretKeySpec` (OWASP M5, CWE-327), ainsi qu'un accès au stockage externe.

3.3.2 Résultats de SecureAPK Analyzer

SecureAPK v1 produit un score de 0/100 (Grade F) avec 27 findings (2 CRITIQUE, 8 ÉLEVÉ, 17 MOYEN). Il détecte `SecretKeySpec` et SHA-1, que MobSF n'identifie pas. En revanche, il ne détecte ni le certificat debug ni les vulnérabilités de stockage externe, deux éléments correctement identifiés par MobSF : ces manques constituent des limites connues à adresser dans les versions futures.

Catégorie de finding	SecureAPK v1	MobSF
<code>debuggable=true</code>	Oui (CRITIQUE)	Oui (ÉLEVÉ)
<code>SecretKeySpec</code>	Oui	Non
Algorithme SHA-1	Oui	Non
Secrets codés en dur	Oui	Oui ($\times 15$)
Logs sensibles	Oui	Oui
Certificat debug	Non	Oui (ÉLEVÉ)
Stockage externe accessible	Non	Oui
Score global	44/100 — CRITIQUE	38/100 — ÉLEVÉ
Temps d'analyse (indicatif)	≈ 10 s	≈ 35 s

Table 3: Comparaison SecureAPK v1 et MobSF — `SecureStorageLabJava.apk`.

3.3.3 Résultats de MobSF

MobSF attribue un score de 38/100 et détecte le certificat debug (ÉLEVÉ), `debuggable`, `minSdk=24`, les logs sensibles, le stockage externe accessible et 15 secrets codés en dur. Le temps d'analyse mesuré est de ≈ 35 secondes (mesure unique, indicative).

3.4 Synthèse comparative

Métrique	SecureAPK v1	MobSF
Test 1 : NuberBook		
Score / Risque	39/100 — CRITIQUE	43/100 — ÉLEVÉ
Findings détectés	32	≈15
Temps d’analyse (indicatif)	≈15 s	≈45 s
Détection <code>debuggable</code>	Oui (CRITIQUE)	Oui (ÉLEVÉ)
Détection <code>cleartextTraffic</code>	Oui (ÉLEVÉ)	Oui (ÉLEVÉ)
Détection <code>checkClientTrusted</code>	Oui	Non
Détection SHA-1/MD5	Oui	Oui
Détection injection SQL	Oui	Partiel
Corrélation comportementale	Heuristique (AIBA)	Non
Test 2 : SecureStorageLabJava		
Score / Risque	44/100 — CRITIQUE	38/100 — ÉLEVÉ
Findings détectés	27	≈12
Temps d’analyse (indicatif)	≈10 s	≈35 s
Détection <code>SecretKeySpec</code>	Oui	Non
Détection SHA-1	Oui	Non
Détection certificat debug	Non	Oui (ÉLEVÉ)
Détection stockage externe	Non	Oui

Table 4: Comparaison SecureAPK v1 et MobSF sur deux APK pédagogiques. Les scores des deux outils ne sont pas directement comparables : leurs échelles de notation et leurs pénalités diffèrent. Les temps d’analyse sont des mesures uniques indicatives, non représentatives d’une moyenne statistique.

L’analyse de ces résultats met en évidence une complémentarité entre les deux outils. SecureAPK v1 excelle dans la détection de `checkClientTrusted`, `SecretKeySpec` et des algorithmes cryptographiques faibles (SHA-1, MD5), absents des résultats MobSF sur ces APK. En revanche, MobSF détecte le certificat debug et les vulnérabilités de stockage externe, deux catégories non couvertes par SecureAPK dans sa version actuelle. La différence de scores (0/100 vs 38–40/100) s’explique par des échelles de notation fondamentalement différentes : les deux outils s’accordent sur un niveau de risque CRITIQUE/ÉLEVÉ pour les deux APK testées.

4 Perspectives futures et limites

4.1 Limites de validité

Les résultats présentés appellent plusieurs réserves méthodologiques. Premièrement, le corpus d’évaluation est limité à deux APK pédagogiques construites pour des travaux pratiques : ces applications ne représentent pas la diversité des applications réelles et les résul-

tats ne peuvent être généralisés sans évaluation sur un corpus annoté de référence (DroidBench [11], AndroZoo [12]). Deuxièmement, les seuils heuristiques du `RiskPredictor` et de l'`AIBehaviorAnalyzer` n'ont pas été calibrés empiriquement ; ils reflètent un jugement d'expert et non une optimisation mesurée sur des données réelles. Troisièmement, les temps d'analyse rapportés sont des mesures uniques indicatives ; une analyse de variance sur plusieurs répétitions et configurations matérielles est nécessaire pour des conclusions robustes. Quatrièmement, les captures d'écran présentées dans la section 3 ne se substituent pas à des artefacts JSON exportés et vérifiables ; ces exports seront fournis dans le dépôt Zenodo.

4.2 Améliorations du moteur d'analyse

Plusieurs évolutions techniques sont prioritaires. L'extension du corpus de règles vise 150+ règles Java avec identification CWE systématique. La détection du certificat debug et des vulnérabilités de stockage externe, identifiées comme manques en section 3, sera intégrée en priorité. L'analyse des bibliothèques natives (`.so`) avec Ghidra [14] ou IDA constituera une extension significative de la couverture. Par ailleurs, une mesure quantitative des taux de faux positifs avant et après filtrage contextuel permettra de valider empiriquement l'efficacité des composants `LibraryFilter` et `ClassContext`.

4.3 Pipeline d'analyse comportementale par paires d'API

Une extension prévue est l'ajout d'un micro-service dédié à la corrélation comportementale par paires d'API (sur le modèle de Quark Engine [7]) et d'un assistant conversationnel optionnel pour l'explication en langage naturel des résultats. Ce pipeline permettrait de détecter des comportements invisibles à l'analyse statique classique, tels que l'enchaînement de deux méthodes Android révélant une intention malveillante (par exemple, `StringBuilder.append()` suivi de `SQLiteDatabase.rawQuery()`, caractéristique d'une injection SQL construite dynamiquement). Ces composants ne sont pas inclus dans la version actuelle de l'outil et constituent une perspective de développement futur.

4.4 Évaluation et validation

Sur le plan évaluatif, une évaluation sur un corpus étendu (DroidBench [11], AndroZoo [12]) avec vérité terrain annotée permettra de calculer des métriques de précision (précision, rappel, F1, exactitude) et une matrice de confusion par catégorie de règle. Une analyse de sensibilité des poids du `RiskPredictor` quantifiera leur impact sur les verdicts. L'intégration de modèles d'apprentissage automatique pour la détection de malwares zero-day nécessitera la constitution d'un jeu de données annoté et la définition de caractéristiques représentatives avant toute évaluation rigoureuse.

4.5 Intégration et déploiement

Une API REST documentée (OpenAPI) permettra l'intégration dans des pipelines CI/CD, par exemple via un endpoint `POST /api/v1/analyze` retournant un rapport JSON structuré compatible avec les plateformes GitLab CI et GitHub Actions. Des visualisations interactives des résultats et un export PDF des rapports sont également prévus. L'analyse dynamique (exécution dans un émulateur Android contrôlé) complétera l'analyse statique pour les comportements ne se manifestant qu'à l'exécution.

5 Conclusions

Cet article a présenté SecureAPK Analyzer, un outil d'analyse statique d'APK Android combinant un moteur à base de règles (80+ règles), une corrélation heuristique comportementale et un score de risque pondéré avec indice de confiance. L'évaluation sur deux APK pédagogiques volontairement vulnérables montre que l'outil détecte des vulnérabilités spécifiques (`checkClientTrusted`, `SecretKeySpec`, algorithmes cryptographiques faibles) non identifiées par MobSF dans nos conditions de test, tout en présentant des manques sur la détection du certificat debug et du stockage externe. Ces résultats sont illustratifs et ne permettent pas de conclure sur une précision générale sans évaluation sur corpus étendu.

Les contributions principales de l'outil sont : un moteur statique avec filtrage contextuel réduisant les faux positifs liés aux bibliothèques tierces ; une corrélation heuristique comportementale inférant des intentions malveillantes à partir de combinaisons de motifs ; un score de risque pondéré avec indice de confiance ; et un mécanisme de repli sur l'analyse brute des fichiers `.dex` pour les APK obfusquées. L'ensemble du code source, les APK de test avec leurs hash SHA-256 et les scripts de reproduction sont archivés sur Zenodo (DOI : [10.5281/zenodo.20356691](https://doi.org/10.5281/zenodo.20356691)) sous licence MIT, garantissant la reproductibilité des expériences présentées.

Les priorités de développement incluent l'évaluation sur corpus étendu avec métriques calculées, l'intégration CI/CD via API REST, l'adressage des manques de détection identifiés, et l'intégration future d'un pipeline de corrélation comportementale par paires d'API. Face à l'escalade des menaces de cybersécurité mobile, SecureAPK Analyzer constitue un outil complémentaire dans l'écosystème d'analyse, à condition que ses limites soient clairement documentées et que ses heuristiques soient progressivement validées sur des corpus représentatifs.

Remerciements

Les auteurs remercient l'École Nationale des Sciences Appliquées de Marrakech (ENSA Marrakech) pour son soutien dans le développement de ce projet.

References

- [1] Abraham, A., *Mobile Security Framework (MobSF)*. GitHub, 2025. Disponible : <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [2] Skylot, *JADX — Dex to Java decompiler*. GitHub, 2025. Disponible : <https://github.com/skylot/jadx>
- [3] Pivotal/VMware, *Spring Boot Reference Documentation*, version 3.x. 2025. Disponible : <https://spring.io/projects/spring-boot>
- [4] OWASP Foundation, *Mobile Security Testing Guide (MSTG)*. 2025. Disponible : <https://owasp.org/www-project-mobile-security-testing-guide/>
- [5] MITRE Corporation, *Common Weakness Enumeration (CWE)*. 2025. Disponible : <https://cwe.mitre.org/>
- [6] Thomas, M., *AndroWarn — Android APK static analysis*. GitHub, 2025. Disponible : <https://github.com/maaaaz/androwarn>
- [7] Quark-Engine Team, *Quark-Engine — Android Malware Analysis Engine*. GitHub, 2025. Disponible : <https://github.com/quark-engine/quark-engine>
- [8] Rish, I., *An empirical study of the naive Bayes classifier*. In *Proceedings of the IJCAI 2001 Workshop on Empirical Methods in Artificial Intelligence*, pp. 41–46, 2001.
- [9] OWASP Foundation, *Mobile Application Security Verification Standard (MASVS)*, version 2.0. 2024. Disponible : <https://mas.owasp.org/MASVS/>
- [10] Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D., McDaniel, P., *FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps*. *ACM SIGPLAN Notices*, vol. 49, n° 6, pp. 259–269, 2014. DOI : <https://doi.org/10.1145/2666356.2594299>
- [11] Arzt, S. et al., *DroidBench — A micro-benchmark suite for Android taint analysis tools*. GitHub, 2014. Disponible : <https://github.com/secure-software-engineering/DroidBench>

- [12] Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y., *AndroZoo: Collecting Millions of Android Apps for the Research Community*. In *Proceedings of MSR 2016*, pp. 468–471, 2016. DOI : <https://doi.org/10.1145/2901739.2903508>
- [13] Arp, D., Spreitzenbarth, M., Hübner, M., Gascon, H., Rieck, K., *Drebin: Effective and Explainable Detection of Android Malware in Your Pocket*. In *Proceedings of NDSS 2014*, 2014. Disponible : <https://www.ndss-symposium.org/ndss2014/drebin>
- [14] NSA Research, *Ghidra — A software reverse engineering (SRE) framework*. GitHub, 2025. Disponible : <https://github.com/NationalSecurityAgency/ghidra>
- [15] Statista Research Department, *Number of smartphone users worldwide from 2016 to 2028*. Statista, 2023. Disponible : <https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/>
- [16] IDC, *Smartphone Market Share — Operating System*. International Data Corporation, 2024. Disponible : <https://www.idc.com/promo/smartphone-market-share/os>
- [17] Kaspersky Lab, *Mobile Malware Evolution 2022*. Securelist, 2023. Disponible : <https://securelist.com/mobile-malware-evolution-2022/109132/>